Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №2

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Адаптивный шаг обучения в однослойном линейном персептроне (метод наискорейшего спуска)»

Выполнил:

Студент 3 курса

Группы ИИ-26

Петручик Д.А.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: изучить алгоритм оптимизации градиентного спуска с использованием адаптивного шага обучения. Реализовать модифицированный персептрон, в котором параметр скорости обучения t вычисляется на основе минимизации квадратичной формы ошибки для каждой итерации. Сравнить скорость сходимости с классическим алгоритмом из Лабораторной работы №1. Вариант сохраняется.

Ход работы:

Задачи лабораторной работы:

1. Модифицировать алгоритм последовательного обучения (из Лаб №1) таким образом, чтобы на каждой итерации t значение α вычислялось автоматически на основе текущего входного вектора x по формул (см раздел 2.9).
2. Применить вычисленный α(t) для обновления весов ij и порогов Tj согласно дельта-правилу.
3. Используя данные своего варианта, провести два эксперимента:
   - Обучение с фиксированным шагом (например, α=0.1 или α=1p).
   - Обучение с адаптивным шагом по Теореме 2.1 (формула 2.36).

Критерий остановки в обоих случаях – достижение заданной суммарной ошибки Es≤Ee.

4. Построить графики обучения Es(p), где p – номер эпохи, для обоих экспериментов на одних осях координат.
5. Выполнить графическую визуализацию разделяющей линии для адаптивного метода.
6. Реализовать режим функционирования сети:
   - пользователь задаёт произвольный входной вектор,
   - сеть вычисляет выходной класс,
   - соответствующая точка отображается на графике,
7. Написать вывод по выполненной работе. Оценить, насколько адаптивный шаг сокращает количество эпох обучения по сравнению с фиксированным. Обязательно сравнение результатов 1 и 2 лабораторных работ.

Допускается применение **математических** и **графических** библиотек
ML-библиотеки и **ML-фреймворки** использовать **нельзя** (например: scikit-learn, TensorFlow, PyTorch - запрещены)

10.

x_1	x_2	e
6	6	0
-6	6	0
6	-6	1
-6	-6	0

x_1, x_2 - входные данные сети, e - эталонные значения

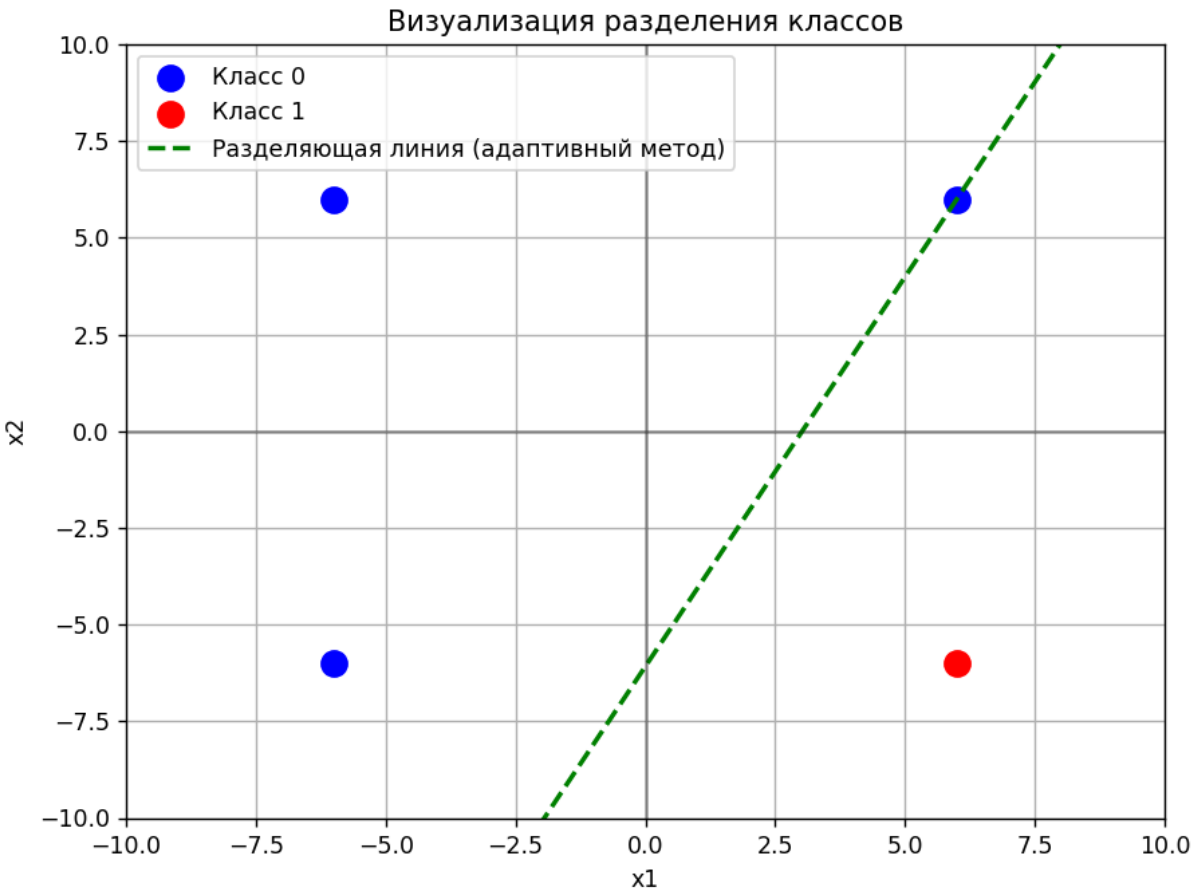
Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([[6, 6], [-6, 6], [6, -6], [-6, -6]] )
e = np.array([0, 0, 1, 0])

target_mse = 0.001
max_epochs = 500
fixed_alpha = 0.01

np.random.seed(42)

def train_perceptron(X_data, e_labels, alpha_type = 'fixed', alpha_val = 0.01) :
    """Обучение персептрона (фиксированный или адаптивный шаг)"""

    w = np.random.uniform(-0.5, 0.5, 2)
    w0 = np.random.uniform(-0.5, 0.5)
    mse_history = []

    for epoch in range(max_epochs) :
        errors = []

        for i in range(len(X_data)) :
            S = np.dot(X_data[i], w) + w0
            delta = e_labels[i] - S

            if alpha_type == 'adaptive' :
                alpha = 1 / (1 + np.sum(X_data[i] * *2))
            else:
alpha = alpha_val

w += alpha * delta * X_data[i]
w0 += alpha * delta

errors.append(delta * *2)

current_mse = np.mean(errors)
mse_history.append(current_mse)

if current_mse <= target_mse:
break

return w, w0, mse_history

w_fixed, w0_fixed, hist_fixed = train_perceptron(X, e, 'fixed', fixed_alpha)
w_adapt, w0_adapt, hist_adapt = train_perceptron(X, e, 'adaptive')

print("Обучение завершено.")
print(f"Фиксированный шаг: {len(hist_fixed)} эпох")
print(f"Адаптивный шаг: {len(hist_adapt)} эпох")

plt.figure(figsize = (10, 5))
plt.plot(hist_fixed, 'r--', label = f'Фиксированный шаг ({len(hist_fixed)} эпох)')
plt.plot(hist_adapt, 'g-', linewidth = 2, label = f'Адаптивный шаг
({len(hist_adapt)} эпох)')
plt.yscale('log')

plt.title("Сравнение скорости сходимости (MSE)")
plt.xlabel("Номер эпохи")
plt.ylabel("Среднеквадратичная ошибка (MSE)")
plt.legend()
plt.grid(True, which = "both", alpha = 0.3)
plt.tight_layout()
```

```
plt.show()

plt.figure(figsize = (8, 6))

for i in range(len(X)) :
    if e[i] == 0 :
        plt.scatter(X[i, 0], X[i, 1], color = 'blue', s = 120,
            label = 'Класс 0' if i == 0 else "")
    else :
        plt.scatter(X[i, 0], X[i, 1], color = 'red', s = 120,
            label = 'Класс 1')

        x_line = np.linspace(-10, 10, 100)

        if w_adapt[1] != 0:
y_line = (0.5 - w0_adapt - w_adapt[0] * x_line) / w_adapt[1]
plt.plot(x_line, y_line, 'g--', linewidth = 2,
    label = 'Разделяющая линия (адаптивный метод)')

    plt.xlim(-10, 10)
    plt.ylim(-10, 10)
    plt.axhline(0, color = 'black', alpha = 0.3)
    plt.axvline(0, color = 'black', alpha = 0.3)
    plt.title("Визуализация разделения классов")
    plt.xlabel("x1")
    plt.ylabel("x2")
    plt.legend()
    plt.grid(True)
    plt.show()

    print("РЕЖИМ ТЕСТИРОВАНИЯ:")

    while True:
user_input = input("\nВведите x1, x2 через запятую (для выхода используйте'q'): ")
if user_input.lower() == 'q' :
    break

    try :
    coords = [float(x.strip()) for x in user_input.split(',')]
    test_point = np.array(coords)

    S = np.dot(test_point, w_adapt) + w0_adapt
    y_class = 1 if S >= 0.5 else 0

    print(f"Точка {coords}")
    print(f"  Сумм S = {S:.4f}")
    print(f"  Класс: {y_class}")

    except Exception as err :
print(f"Ошибка ввода! Введите два числа. ({err})")
```

Результат программы:

График с разделяющей:



```
Обучение завершено.

Введите x1, x2 через запятую (для выхода используйте'q'): 6,6
Точка [6.0, 6.0]
  Сумм S = 0.5000
  Класс: 0

Введите x1, x2 через запятую (для выхода используйте'q'): -6,6
Точка [-6.0, 6.0]
  Сумм S = -0.4932
  Класс: 0

Введите x1, x2 через запятую (для выхода используйте'q'): 6,-6
Точка [6.0, -6.0]
```

Вывод: Проделав данную лабораторную работу я изучил алгоритм оптимизации градиентного спуска с использованием адаптивного шага обучения. Реализовал модифицированный персептрон, в котором параметр скорости обучения t вычисляется на основе минимизации квадратичной формы ошибки для каждой итерации.